

APLICACIONES DISTRIBUIDAS

Coordinación y Consistencia en Sistemas Distribuidos

CAP (Consistency-Availability-Partition) · **PACELC** (Partition→A/C, Else→L/C) · **2PC** (Two-Phase Commit) · Raft · Paxos

Unidad 1 · Semana 4 | 25 – 31 de Mayo del 2026

 **Resultado de Aprendizaje:** Analiza las características y principios de los SD (*Sistemas Distribuidos*), evaluando sus ventajas, limitaciones y retos desde una perspectiva crítica e interdisciplinaria.



Glosario de Siglas y Acrónimos — Semana 4

Todas las siglas usadas en esta sesión — expandidas y explicadas

SD = Sistema Distribuido

Colección de nodos autónomos que aparecen al usuario como uno solo

CAP = Consistency · Availability · Partition tolerance

Teorema: imposible garantizar los 3 simultáneamente en SD

CP = Consistency + Partition tolerance

El sistema prefiere consistencia; puede rechazar peticiones

AP = Availability + Partition tolerance

El sistema siempre responde; datos pueden estar desactualizados

PACELC = Partition→Availability/Consistency, Else→Latency/Consistency

Extensión del CAP que cubre operación normal (sin partición)

PA/EL = Partition=Availability, Else=Latency

Ej: Cassandra, DynamoDB — siempre rápido

PC/EC = Partition=Consistency, Else=Consistency

Ej: HBase, ZooKeeper — nunca dato desactualizado

2PC = Two-Phase Commit (Commit en Dos Fases)

Protocolo de atomicidad distribuida con coordinador central

3PC = Three-Phase Commit (Commit en Tres Fases)

Extiende 2PC con fase PRE-COMMIT para evitar bloqueo

SPOF = Single Point of Failure (Punto Único de Fallo)

Componente cuyo fallo detiene todo el sistema

WAL = Write-Ahead Log (Log de Escritura Anticipada)

Registro de transacciones previo al commit en disco

ACID = Atomicity, Consistency, Isolation, Durability

Propiedades de las transacciones de bases de datos relacionales

RPC = Remote Procedure Call (Llamada a Procedimiento Remoto)

Invocar una función en otro proceso como si fuera local

gRPC = Google Remote Procedure Call

Implementación moderna de RPC con Protocol Buffers y HTTP/2

TCP Transmission Control Protocol · **UDP** User Datagram Protocol · **HTTP** HyperText Transfer Protocol · **JSON** JavaScript Object Notation · **CSV** Comma-Separated Values · **DOI** Digital Object Identifier · **DPI** Dots Per Inch

Agenda — Semana 4

01

Consistencia — CAP y PACELC

Consistency-Availability-Partition | Partition→A/C, Else→L/C

02

Algoritmos de Coordinación

Elección de líder: Bully y Ring · Exclusión mutua distribuida

03

Transacciones — 2PC

Two-Phase Commit · SPOF · WAL · Alternativas: 3PC, Saga, Raft

04

Consenso: Raft y Paxos

Quórum ($n/2+1$) · Replicación de log · Uso en etcd, CockroachDB

⚠ HOY también: GA-SUM-01 (Guía de Aprendizaje Sumativa) · PE-U1 (Práctica Experimental Unidad 1) — entrega y revisión

01 · Consistencia — CAP (Consistency · Availability · Partition tolerance)

Tres modelos de consistencia — cuánto garantiza el SD sobre el valor que devuelve:

Linearizabilidad (Consistencia fuerte)

Cada operación parece ejecutarse atómicamente. Como si existiera un único servidor. Todos los clientes ven el mismo valor al mismo tiempo.

Costo: Alta latencia — requiere consenso (Raft/Paxos)

Ej: CockroachDB, Google Spanner, ZooKeeper

Consistencia causal

Operaciones causalmente relacionadas se ven en el mismo orden. Concurrentes pueden verse diferente en nodos distintos.

Costo: Latencia media — requiere relojes vectoriales

Ej: MongoDB (causal session), DynamoDB (conditional writes)

Consistencia eventual

Si no hay más escrituras, todas las réplicas convergen al mismo valor eventualmente. Sin garantía de cuándo.

Costo: Baja latencia — alta disponibilidad

Ej: Cassandra, DynamoDB (default), CouchDB

Teorema CAP (Brewer, 2000 — demostrado por Gilbert & Lynch, 2002)

P (Partition) no es opcional — las redes siempre pueden fallar:

CP = C+P: bloquear hasta consistencia (HBase, CockroachDB)

AP = A+P: responder aunque datos sean viejos (Cassandra)

CA = C+A: solo con 1 nodo — imposible en SD real

PACELC (Abadi, 2012) — mejora del CAP

If **P** Partition: elegir **A** Availability OR **C** Consistency

Else: elegir Latency OR Consistency

PA/EL: Cassandra, DynamoDB

PC/EC: HBase, ZooKeeper

PC/EL: CockroachDB, Spanner

02 · Algoritmos de Coordinación — Elección de Líder

¿Por qué un líder? En un SD (*Sistema Distribuido*) sin coordinador central, los procesos no pueden ponerse de acuerdo en el orden de operaciones. Un líder activo resuelve esto: si cae, los nodos eligen uno nuevo automáticamente vía **heartbeats** (latidos de red — mensajes periódicos que indican que el líder sigue activo).

Algoritmo Bully (García-Molina, 1982)

Premisa: Cada nodo tiene un ID numérico. El nodo con mayor ID es el líder.

Paso 1: Nodo P detecta caída del líder (timeout del heartbeat — sin respuesta en T ms).

Paso 2: P envía mensaje ELECTION a todos los nodos con ID > P.

Paso 3: Si ninguno responde OK → P se proclama líder, envía COORDINATOR a todos.

Paso 4: Si alguno responde OK → ese nodo con mayor ID reinicia el proceso.

Problema: Si el nodo de mayor ID se recupera, interrumpe al líder vigente → 'agresivo'.

Complejidad: $O(n^2)$ mensajes — n = número de nodos en el clúster.

Algoritmo Ring (Chang-Roberts, 1979)

Premisa: Nodos organizados en anillo lógico (no físico) ordenado por ID.

Paso 1: Cualquier nodo detecta la caída e inicia ELECTION enviando su ID al vecino derecho.

Paso 2: Cada nodo compara el ID recibido con el suyo; pasa el mayor al siguiente.

Paso 3: Cuando el ID del iniciador da la vuelta completa → él es el líder elegido.

Paso 4: El nuevo líder envía COORDINATOR al anillo para notificar a todos.

Ventaja: $O(n)$ mensajes — n veces más eficiente que Bully.

Problema: Si el anillo se rompe (nodo intermedio cae), el mensaje ELECTION se pierde.

 **Uso industrial:** ZooKeeper (Paxos-like) · etcd (*Kubernetes*) → Raft · MongoDB replica sets → Bully variant · Redis Sentinel → quórum

03 · 2PC — Two-Phase Commit (Commit en Dos Fases)

Problema: garantizar **atomicidad** (todo-o-nada) cuando una transacción toca MÚLTIPLES nodos simultáneamente

FASE 1 — Preparación (Voting Phase / Fase de Votación)

1. Coordinador envía PREPARE a todos los participantes.
2. Cada participante: ejecuta la transacción localmente SIN commit, escribe en **WAL** (Write-Ahead Log — registro previo a disco), responde VOTE-YES o VOTE-NO.
3. Si alguno vota NO o hay timeout → coordinador decide ABORT global.

FASE 2 — Decisión (Commit o Abort / Confirmar o Revertir)

4. Si TODOS votaron YES → coordinador envía COMMIT (confirmar).
5. Cada participante hace efectivo el commit en disco, libera bloqueos (locks) de filas/tablas, escribe COMMITTED en el WAL.
6. Si alguno votó NO → ABORT. Todos hacen rollback (revertir cambios) y liberan bloqueos.

⚠ Problemas críticos del 2PC (Two-Phase Commit)

SPOF (Single Point of Failure = Punto Único de Fallo): si el coordinador cae ENTRE las dos fases, los participantes quedan bloqueados esperando.

Protocolo bloqueante: participantes mantienen locks (bloqueos) hasta recibir la decisión. Alta concurrencia → deadlocks.

No tolera particiones: si la red se parte entre Fase 1 y Fase 2, el sistema no puede avanzar.

✓ Alternativas modernas al 2PC

3PC (Three-Phase Commit): añade fase PRE-COMMIT para evitar bloqueo. Complejo, poco usado en producción.

Saga Pattern: pasos locales con transacciones compensatorias ("undo" por fallo). Netflix, Uber, Amazon. Favorece AP.

Raft/Paxos: consenso distribuido con quórum (*quórum = mayoría $n/2+1$*). Garantiza progreso aunque fallen nodos.

04 · Sistemas de Consenso — Raft y Paxos

Consenso: N procesos deben acordar UN valor aunque algunos fallen. Requisito: **quórum** (quórum = mayoría simple, mínimo $n/2+1$ nodos activos). Si $\geq$ quórum están activos, el sistema avanza.

Paxos (Lamport, 1989/1998)

Roles:

- Proposers (Proponentes): proponen valores al sistema
- Acceptors (Aceptantes): aceptan o rechazan propuestas
- Learners (Aprendices): aprenden el valor finalmente elegido

Fase 1 — Prepare/Promise:

Proposer envía PREPARE(n). Acceptors prometen no aceptar propuestas con número de ronda n menor.

Fase 2 — Accept/Chosen:

Si quórum promete → proposer envía ACCEPT(n, valor). Si quórum acepta → valor elegido (chosen).

Reputación: difícil de implementar. Usado en Google Chubby (2006) y Google Megastore.

Raft (Ongaro & Ousterhout, 2014)

Diseñado para ser comprensible — equivalente a Paxos.

Elección de líder:

- Terms (Términos = períodos de liderazgo numerados). Sin heartbeats → timeout → follower (*seguidor*) se convierte en candidate (*candidato*) → pide votos → quórum → nuevo leader (líder).

Replicación de log (registro de operaciones):

- El líder recibe escrituras → añade al log → replica a followers. Entrada 'committed' cuando quórum confirma recepción.

Seguridad:

- Solo 1 líder por term. Un nodo solo vota por candidatos con log más actualizado.

Usado en:

etcd (*Kubernetes*) · CockroachDB · TiKV (*Ti Key-Value*) · Consul · TiDB

 **Ver ahora:** raft.github.io — animación interactiva de Raft (5 min). Mostrar: (1) heartbeats normales, (2) matar el líder y ver re-elección, (3) partición de red: el grupo SIN quórum no puede elegir líder.

CockroachDB: cada rango de datos tiene su propio grupo Raft de 3 nodos. Escritura necesita quórum (2 de 3). En Semana 11, detener un nodo → Raft re-elegirá líder automáticamente → quórum sigue activo.

GA-SUM-01 (*Guía de Aprendizaje Sumativa N° 01*) • **PE-U1** (*Práctica Experimental Unidad 1*)

Comunicación entre Procesos Distribuidos con gRPC (*Google Remote Procedure Call*) **y** **Sockets TCP** (*Transmission Control Protocol*)

Campo	Detalle
Código	GA-SUM-01 (GA = Guía de Aprendizaje · SUM = Sumativa)
Unidad	1 — Generalidades de los SD (Sistemas Distribuidos)
Tipo	Aplicación de contenidos procedimentales
Modalidad	Grupal (3-4 estudiantes)
Duración	3 horas laboratorio + 5 horas autónomas
Entregable	Repositorio GitHub + Documento LaTeX compilado (PDF + .tex)
Semana de entrega	4 — 25 al 31 de Mayo del 2026

Estructura obligatoria del repositorio GitHub

```
/src/sockets/server.py · /src/sockets/client.py
/src/grpc/messaging.proto · /src/grpc/server_grpc.py · /src/grpc/client_grpc.py
/data/latency_sockets.csv · /data/latency_grpc.csv (CSV = Comma-Separated Values)
/docs/guia1_practica.tex · /docs/guia1_practica.pdf · /docs/figures/boxplot.png (DPI = Dots Per Inch)
README.md — instrucciones completas de ejecución
```

 **LaTeX que NO compila con pdflatex = 0 pts en ese criterio. Verificar localmente o en Overleaf ANTES de entregar.**

GA-SUM-01 · Los 5 Pasos del Procedimiento

Paso 1: Entorno (Environment)

- ▶ `pip install grpcio grpcio-tools protobuf pandas matplotlib`
- ▶ Crear repo GitHub con estructura requerida (repo = repositorio)
- ▶ 3 procesos: `localhost:5000` (server), `:5001`, `:5002` (clients)
- ▶ Cada integrante: ≥ 3 commits significativos en el historial de Git

Paso 2: Sockets TCP (Transmission Control Protocol)

- ▶ Servidor: `socket.socket(AF_INET, SOCK_STREAM) · bind(5000) · listen()`
- ▶ Cliente: implementar `LamportClock · tick()` antes de enviar · `receive(ts)` al recibir
- ▶ Payload JSON (JavaScript Object Notation): `{sender, timestamp, message}`
- ▶ 20 intercambios + 100 mediciones con `time.perf_counter()` → guardar CSV

Paso 3: gRPC (Google Remote Procedure Call)

- ▶ Definir `messaging.proto` (MessagingService + Request/Response)
- ▶ Generar: `python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. messaging.proto`
- ▶ Misma `LamportClock` integrada en el servicio gRPC
- ▶ 20 intercambios + 100 mediciones → guardar CSV

Paso 4: Análisis Estadístico

- ▶ pandas: media, mediana, std (desviación estándar), P95 (percentil 95)
- ▶ matplotlib: boxplot comparativo (300 DPI — Dots Per Inch) en `/docs/figures/`
- ▶ ¿Cuál tiene menor latencia? ¿Cuál menor variabilidad (std)? ¿Por qué?
- ▶ ¿En qué escenario del PFC (Proyecto Fin de Curso) usarías Sockets TCP y en cuál gRPC?

Paso 5: Documento LaTeX

- ▶ Portada · Resumen bilingüe · Fundamento teórico U1 completo
- ▶ Diagrama TikZ · `Istlisting` (código) · tabla booktabs + boxplot con `\ref{}`
- ▶ ≥ 6 referencias IEEE (Institute of Electrical and Electronics Engineers) con DOI (Digital Object Identifier)
- ▶ Compilar con `pdflatex` sin errores · subir PDF + `.tex` a `/docs/`

✓ Antes de entregar: repo público · PDF en `/docs/` · ≥ 3 commits/integrante · README con run instructions

lamport_clock.py — LC = Lamport Clock (Reloj Lógico de Lamport)

```
class LamportClock:
    def __init__(self, pid): # pid = Process ID
        self.pid = pid; self.lc = 0
    def tick(self):          # Reglas 1 y 2
        self.lc += 1        # LC = LC + 1
        return self.lc
    def receive(self, ts): # Regla 3: ts = timestamp
        self.lc = max(self.lc, ts) + 1
        return self.lc
    def __repr__(self):
        return f"P{self.pid}[LC={self.lc}]"
```

messaging.proto — Buffers Protocol

```
syntax = "proto3"; // proto = Protocol Buffers v3
package messaging;
service MessagingService {
    rpc SendMessage(Request) // RPC = Remote Procedure Call
        returns (Response); }
message Request {
    string sender      = 1; // campo 1
    int64  timestamp  = 2; // campo 2: entero 64 bits
    string content     = 3; // campo 3
}
message Response {
    string status      = 1;
    int64  server_ts  = 2; // ts = timestamp
}
```

benchmark.py + análisis.py — std = standard deviation (desviación estándar) · P95 = percentil 95

```
import time, csv, pandas as pd, matplotlib.pyplot as plt

def measure(send_fn, n=100): #  $\Delta$  perf_counter, NO time.time()
    return [(lambda t0, _: (time.perf_counter()-t0)*1000)(time.perf_counter(), send_fn()) for _ in range(n)]
def save_csv(data, fname): # CSV = Comma-Separated Values
    pd.DataFrame({"latency_ms":data}).to_csv(fname, index=False)
def analyze(sockets_csv, grpc_csv):
    df = pd.DataFrame({"Sockets TCP":pd.read_csv(sockets_csv)["latency_ms"],
                      "gRPC":pd.read_csv(grpc_csv)["latency_ms"]})
    print(df.describe(percentiles=[.5,.95])) # std, P50 (mediana), P95
    df.boxplot(); plt.savefig("docs/figures/boxplot.png", dpi=300); plt.show() # dpi = Dots Per Inch
```

Resumen Semana 4 + Preguntas Obligatorias GA-SUM-01



Consistencia

CAP (C·A·P) → CP o AP durante partición. PACELC (P→A/C, Else→L/C) añade trade-off latencia vs. consistencia en operación normal.



Elección de Líder

Bully $O(n^2)$ — heartbeats, ID mayor gana · Ring $O(n)$ — anillo lógico. Usados en etcd, ZooKeeper, MongoDB.



2PC (Two-Phase Commit)

Atomicidad distribuida. Problema: bloqueante + SPOF (Single Point of Failure). Alternativas: Saga, 3PC, Raft.



Raft / Paxos

Consenso con quórum ($n/2+1$). Raft: terms + log replication. Usado en etcd (Kubernetes), CockroachDB, TiKV.



Preguntas Obligatorias GA-SUM-01 — responder en el documento LaTeX con datos propios

1. ¿Cómo afecta el número de nodos al tiempo de convergencia de los relojes Lamport? Justifica con datos medidos.
2. ¿En qué escenario del PFC (Proyecto Fin de Curso) usarías sockets TCP y en cuál gRPC? Argumenta desde el teorema CAP.
3. Si un nodo falla durante la comunicación, ¿qué ocurre con los timestamps (marcas temporales) de Lamport en los nodos restantes?
4. ¿Qué transparencias ANSA (Architecture for Networks of Systems and Applications) logra el sistema? ¿Cómo implementarías las faltantes?